Scanz

Payment failed.
The investigation starts now.

A checkout support teammate for merchants and their customers
Hackathon prototype · Simulated payment partners

The payment mismatch at checkout



799

Bank: debited
Payment app: failed
Cashier: payment unconfirmed

The customer is still in the queue.

Scanz checks the original payment before anyone repeats a charge or requests a reversal.

Customer and merchant perspectives

CUSTOMER

“Did I pay?
Should I pay again?”

Select the transaction.
Follow one saved request.
See credit only when acknowledged.

MERCHANT / CASHIER

“Did we receive it?
Can checkout continue?”

Inspect merchant-side evidence.
Avoid duplicate financial actions.
Review specific exceptions.

Three outcomes for the same reported problem

Evidence	Teammate action	Verified outcome
Merchant received payment	Return payment acknowledgement	Merchant payment confirmed
Debit + final merchant non-credit	Request authorized reversal	Credit confirmed only with bank ack
Merchant result remains unknown	Keep checking, then request review	Open case; no guessed refund

A customer-reported debit alone never authorizes a reversal.

The customer journey in Scanz

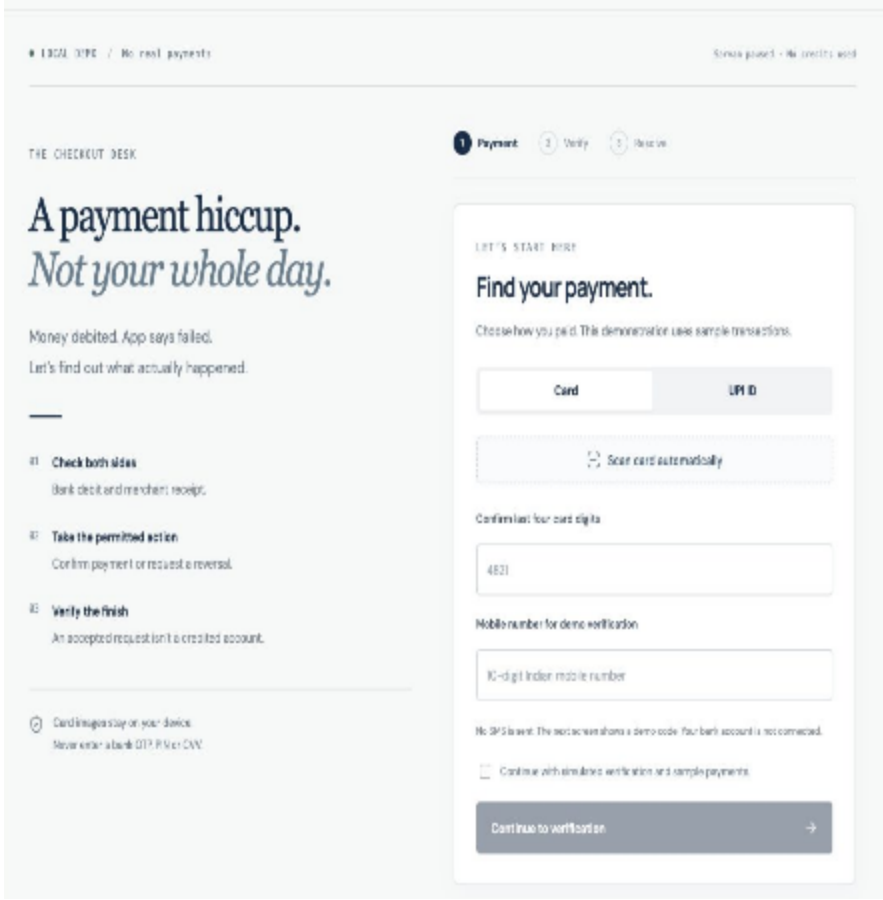
01 Scan card or enter UPI ID

02 Complete demo verification

03 Select the exact sample payment

04 Choose status help or recovery

05 Follow saved evidence to an outcome

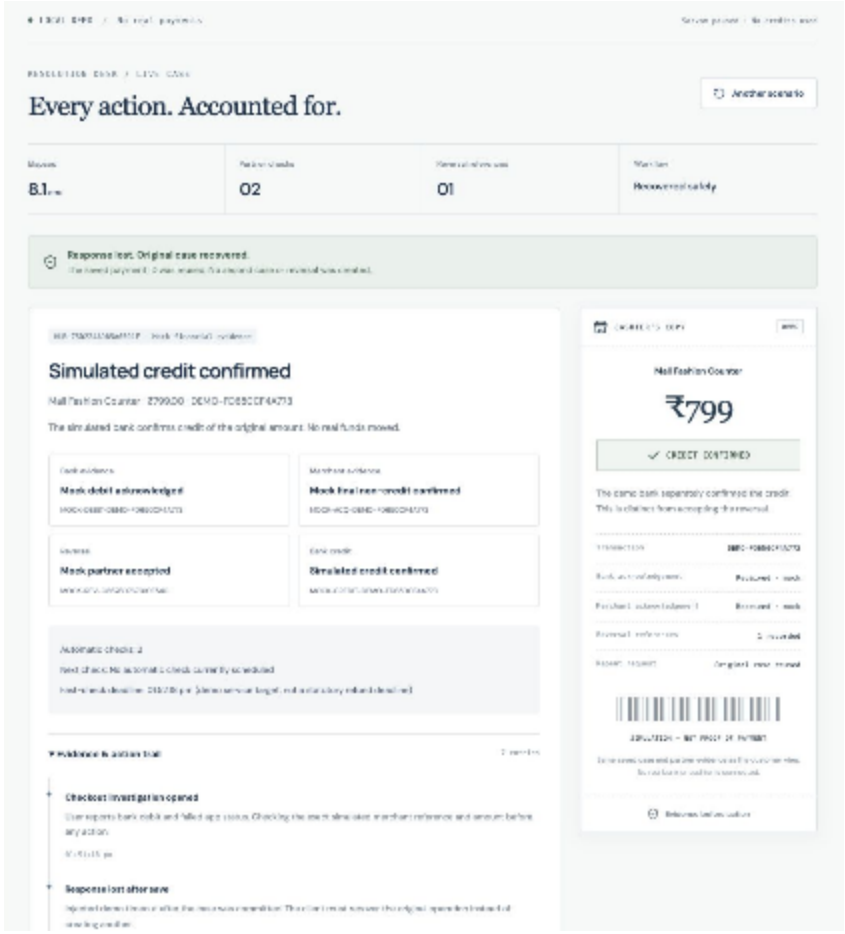


Recovery after a lost response

8.1 sec

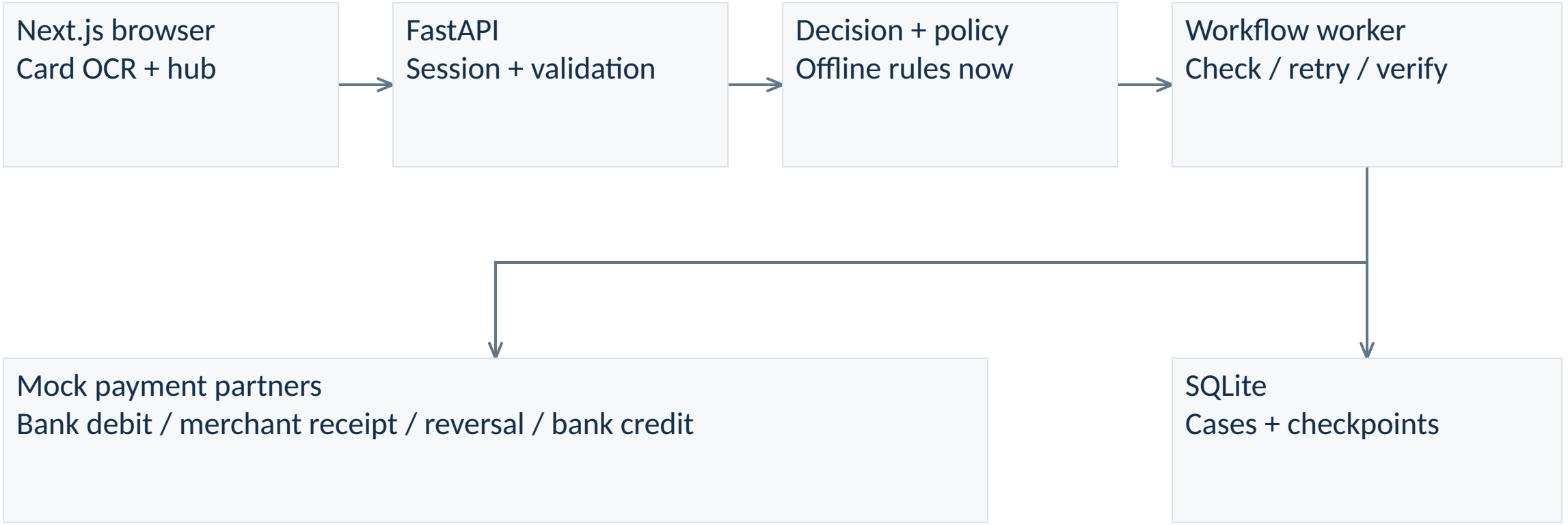
One recorded simulated run

Case saved → response lost
Retry → original case recovered
One mock reversal reference
Separate simulated credit acknowledgement



Full-stack architecture

Sarvam adapter prepared; live model calls paused



Evidence returns to the hub; the page shows state, next check and acknowledgement references.

Tech stack and responsibilities

Layer	Technology	Responsibility
Web interface	Next.js · React · TypeScript	Customer journey and evidence views
Card reader	OpenCV · Tesseract.js	Local image processing and OCR
Backend	FastAPI · Pydantic	APIs, ownership and policy checks
Workflow + storage	Python worker · SQLite	Scheduled checks and saved checkpoints
AI adapter	Sarvam — currently paused	Complaint interpretation and tool selection

What makes the teammate useful

It does work across records

Check payment evidence apply policy take the permitted action verify the result

It continues when systems fail

Saved checkpoints, bounded checks and original-operation reuse prevent a lost response from becoming a duplicate action.

Hackathon demonstration

1. Open a fresh checkout scenario
2. Enable “Test a system failure”
3. Resolve and inspect the cashier copy
4. Reopen the saved case from My requests

Show the uncertain case too: unresolved credit must remain open.

Prototype boundaries and production path

Working in this prototype	Needed before real financial use
Local scanning and scoped demo sessions	Authorized account/customer verification
Mock bank/acquirer acknowledgements	Contracted partner APIs + signed callbacks
Policy, retries and saved evidence	Permitted reversal/refund authority
Simulated credit confirmation	Actual bank-credit acknowledgement
Prepared Sarvam adapter, paused	Budgeted live-model evaluation

The pitch to remember

“Money debited, app failed”
should start an investigation
while the customer is still there.

Scanz coordinates the evidence, the permitted action and the follow-through.